



p5.js_cheat_sheet_updated

p5.js website <https://p5js.org/>

openprocessing website <https://openprocessing.org/>

▼ Template

▼ HTML

```
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
  <!-- cdn: content delivery networkを用いる、もしくはp5.min.jsかp5.jsをscriptタグで読み込む-->
  <script src="./p5.min.js"></script>
  <script src="./p5.sound.min.js">>
  <script src="./particle.js"></script>
  <!-- type='module'、importを使った場合p5.jsが機能しなくなるため避ける -->
  <!-- 変数名の重複には注意 -->
  <script src="./index.js"></script>
  <title>p5.js Project</title>
</head>

<body>
  <section id="p5_section">
  </section>
</body>

</html>
```

▼ CSS

```
/* 共通の初期設定 => */
*{
  margin: 0;
  padding: 0;
}

body{
  background-color: #333;
}
```

```
#p5_section{
  width: 100vw;
  height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
}
/* <= 共通の初期設定 */
```

▼ JS

```
let canvas;
let img, sound;

function preload() {  // 画像ファイルや音声ファイルのプレロード
  img = loadImage('./something.png');
}

function windowResized() {  // リサイズイベントへの対応
  setup();
}

function setup() {  // 初期化。一度だけ実行される
  frameRate(30);  // 1秒間に表示されるフレーム数を指定(デフォルトで60)
  canvas = createCanvas(windowWidth*9/10, windowHeight*9/10);  // p5.js用の
  キャンバスを作成
  p5Canvas.parent('p5_section');  // canvasをhtmlファイル内の section#'p5_sec
  tion' の子要素に指定
  background(100);
}

function draw() {  // frameRateの頻度で上書き描画
  background(100);  // 色については別項目を参照
}
```

▼ draw()の設定

▼ フレームレート

```
function setup(){
  //
  frameRate(30);  // 1秒間に30回フレームが切り替わる (drawが実行される)
}
```

▼ ループの制御[loop(), noLoop(), redraw()]

▼ 1度のみdraw()を実行

```
function setup() {
  //
  noLoop(); // draw()を1度実行して停止
}
```

▼ 一定時間後に停止

```
function draw() {
  //
  if(120 < frameCount){
    noLoop();
  };
}
```

▼ クリックによるコマ送りの例

```
function mousePressed() { // マウスクリックイベントリスナー
  //
  redraw(); // 1度draw()を実行
}
```

▼ マウスプレス・リリースによるループの切り替えの例

```
function mousePressed() { // 停止
  noLoop();
}
function mouseReleased() { // 再開
  loop();
}
```

▼ システム変数

▼ 幅や高さ

```
windowWidth, windowHeight

width, height // canvasの幅、高さ

displayWidth, displayHeight // ディスプレイの幅、高さ
```

▼ フレームカウント、時間関連

```
frameCount // その時点までのフレームカウント  
  
deltaTime // フレームが更新される時間(30fpsの場合ならば、1/30秒)
```

▼ マウスポジション

▼ 単純なマウスのポジション

```
mouseX  
mouseY
```

▼ 1フレーム前のマウスのポジション

```
pmouseX  
pmouseY  
// line(pmouseX, pmouseY, mouseX, mouseY)で途切れのない線描が可能
```

▼ Windowに対するマウスのポジション

```
winMouseX  
winMouseY
```

▼ Windowに対する1フレーム前のマウスのポジション

```
pwinMouseX  
pwinMouseY
```

▼ モード設定の保存、復元

```
push();  
// fill("#ff0"), noStroke(), imageMode(CENTER), colorMode(RGB), ellipseMode(CORNE  
R)など  
// transformationのtranslateやscaleなど  
  
pop(); // pop()によりpush()以降の設定は解除され、push()以前の設定に復元される
```

▼ 図形描画に関する設定（カラー、切り抜き、塗りつぶし、線の幅など）

▼ カラー

▼ モード

```
colorMode(MODE); // MODEの種類はRGB、HSB(HSL)
colorMode(MODE, max1, max2, max3, [maxA]) // 設定数値の最大値を指定することも可能

colorMode(RGB, 255, 255, 255, 1);
colorMode(HSB, 360, 100, 100, 100);
```

▼ カラーの作成[color()]

```
color(50, 100, 150, 0.5); // colorMode(RGB, 255, 255, 255, 1)の設定の場合
color("#3399aa") or color("#39a"); // 16進数、ヘキサデシマル
color("rgba(100, 200, 255, 0.8)"); or color("rgba(50%, 20%, 30%, 0.5)");
// "rgba()", "rgb()"でも可
color("red") // その他"green"など

const newColor = color("#f00");
newColor.setAlpha(0.5); // alphaを追加指定。const newColor = color("#f00").setAlpha(0.5)のようなチェーンは不可
```

▼ 2色の中間色などを抽出[lerpColor()]

```
const from = color("red");
const to = color("blue");
const colorA = lerpColor(from, to, 3/10); // fromからtoに移行する3/10地点の色
```

▼ 切り抜き、透明化

```
// erase([strengthFill], [strengthStroke])

// shape1を描画 // 画面奥の図形
erase(); // erase(255, 255)に等しい。数値を下げると透明化
// shape2を描画; // 画面手前の図形
noErase();
// 背景とshape1のshape2部分を切り抜き[erase(), noErase()]
// shape2部分はbodyの色になる
```

▼ colorMode(HSL)を利用したランダムカラーの作成例

```
colorMode(HSL); // 360, 100, 100, 1
let colour = [floor(random()*360), 100, 50];
stroke(colour);
```

▼ 塗りつぶし

```
fill(COLOR);    // 円や矩形の内部をCOLORで塗りつぶす指定

noFill();       // 円や矩形の内部を塗りつぶさない
```

▼ 線、外周線

```
stroke(COLOR);  // 線、外周線をCOLORに
noStroke();     // 線、外周線を描画しない
strokeWeight(20); // 線、外周線の太さ
strokeCap(ROUND); // 線、外周線の末端の形状を指定。SQUARE, PROJECT
strokeJoin(MITER); // 線、外周線の連結の形状を指定。BEVEL, ROUND
```

▼ 背景、基本図形

▼ 頂点を指定して直線でつなぐ

```
beginShape();
  vertex(x0, y0); // 頂点を定める
  // ...
  vertex(x10, y10);
endShape(); // endShape(CLOSE)の場合は始点と終点をつなぐ
```

```
// 詳しい実例は => https://p5js.org/reference/#/p5/beginShape

// beginShape(SHAPE)
// POINTS : Draw a series of points
// LINES : Draw a series of unconnected line segments (individual lines)
// TRIANGLES : Draw a series of separate triangles
// TRIANGLE_FAN : Draw a series of connected triangles sharing the first vertex in a fan-like fashion
// TRIANGLE_STRIP : Draw a series of connected triangles in strip fashion
// QUADS : Draw a series of separate quad
// QUAD_STRIP : Draw quad strip using adjacent edges to form the next quad
```

▼ 背景

```
background(COLOR);
background(IMAGE); // IMAGEはロードしたイメージファイル
```

```
// 背景の描画を利用して残像を残す
background(COLOR, 0.2); // 背景の塗りつぶしにアルファを適用することで、その他の図形の
// 以前の姿が残り残像となる
```

▼ 円、楕円、円弧

▼ モード

```
ellipseMode(MODE); // MODEには以下のものを設定。デフォルトはCENTER
// CENTER: cx, cy, halfW, halfH
// RADIUS: cx, cy, halfW, halfH
// CORNER: upperLeftX, upperLeftY, w, h
// CORNERS: upperLeftX, upperLeftY, bottomRightX, bottomRightY
```

▼ 円[ellipse(cX, cY, w, h)]

```
// ellipseMode(MODE)を使って円、楕円を描く

ellipseMode(); // デフォルトのCENTER
ellipse(100, 100, 60, 30);
```

▼ 円[circle(cX, cY, diameter)]

```
circle(100, 100, 20); // 半径は10
```

▼ 円弧、円の一部[arc(cX, cY, w, h, startAngle, stopAngle, MODE)]

```
arc(50, 100, 30, 40, HALF_PI, PI); // 時計の3時(0)から $\pi/2$ 進んだ6時から $\pi$ 進んだ12時までの半円
// MODE
// 指定なし : 中心と円弧の両端を結ぶ(strokeなし)
// PIE : 中心と円弧の両端を結ぶ(strokeあり)
// OPEN : 円弧の両端を結ぶ(strokeなし)
// CHORD : 円弧の両端を結ぶ(strokeあり)
```

▼ 点[point(x, y)]

```
strokeWeight(10); // 点の大きさはstrokeWeightで指定
point(50, 50);
```

▼ 線[line(fromX, fromY, toX, toY)]

```
strokeWeight(5);  
line(100, 100, 200, 200);
```

▼ 矩形[rect(x, y, w, h, cornerRadius)]

▼ モード

```
rectMode(CENTER);  
// CORNER : rect(x, y, w, h) // default  
// CENTER : rect(cX, cY, w, h)  
// CORNERS : rect(x1, y1, x2, y2) // 対角の座標  
// RADIUS : rect(cX, cY, halfW, halfH)
```

▼ 矩形

```
rectMode(CENTER);  
rect(30, 20, 100, 50, 10); // (30, 20)を中心にして、幅100、高さ50、角の丸みの  
半径10
```

▼ 三角形[triangle(x1, y1, x2, y2, x3, y3)]

```
triangle(100, 0, 80, 30, 120, 30); // 3点を頂点とした三角形
```

▼ イメージ

```
image(PATH or LOADED_IMAGE, destinationX, destinationY, destinationWidth, destinationHeight,  
[subsectionX, subsectionY,] [subsectionWidth, subsectionHeight]);  
// 直接パスを入力するかロードしたイメージの変数  
// destination: canvas上のイメージ描画の起点座標、描画させる幅と高さ  
// subsection: 元のイメージを切り取る場合の起点座標（左上）、切り取る幅と高さ。スプライトシート  
利用時などに使う
```

▼ モード

```
imageMode(MODE);  
// MODE  
// CORNER: (x, y, w, h) // デフォルト。w, hは省略可能  
// CORNERS: (x1, y1, x2, y2) // この中に収めるように配置  
// CENTER: (cX, cY, w, h) // イメージの中心を指定。w, hは省略可能
```



```
imageMode(CENTER)
image("./something.png", 50, 100);    // (50, 100)を中心にしてオリジナルサイズで描画
```

▼ リサイズ

```
IMAGE.resize(w, h);    // IMAGEはロードされた画像
```

▼ テキスト

```
FONT = loadFont('./abc.otf')    // preloadで

textFont(FONT);

textStyle(NORMAL);    // ITALIC, BOLD, BOLDITALIC
textSize(FONTSIZE);    // pixels
textAlign(CENTER, CENTER);    // horizontal: RIGHT, CENTER, LEFT, vertical: TOP, C
ENTER, BOTTOM, BASELINE

textWrap(WORD);    // WORD:単語単位で、CHAR:文字単位で折り返す。この場合、以下のように範囲を
指定する
text('text', x, y, w, h);    // rectMode()の設定も必要。rect(x, y, w, h)に対して、textA
lign()の設定となる。    分かりにくいので実際に試すのがよい
```

▼ 便利な関数

▼ Math関連

▼ ランダム random

```
random();    // 0以上1未満
random(3);    // 0以上3未満
random(-5, 5);    // -5以上5未満
random(ARRAY);    // ARRAYからランダムチョイス
```

▼ ある値を指定の範囲の値に変換 map

```
// 例) マウスのX座標、0~widthを0~2πに変換する
let angle = map(mouseX, 0, width, 0, Math.PI * 2, true);
// 最後のargumentをtrueにすることで、0-widthを越えた値は対象にしない。
```

▼ 内分点 lerp(start, stop, amt)

```
lerp(start, stop, amt); // amtは0〜1の範囲。0ならばstart、1ならばstop。0.5ならばstartとstopの中間
// (stop - start) * amt + start
```

▼ 正規化 (0-1) に変換 norm

```
norm(value, start, stop); // startは下限 (最小) の数、stopは上限 (最大) の数。
その範囲でのある値value
```

▼ 上限と下限を設定 constrain

```
constrain(n, low, high);
// n (変数) の値をlow以上highいかに制限

// 例) mouseXの値を100〜300に制限
let x = constrain(mouseX, 100, 300); // mouseXが100より小さくなくても、xの値
は下限の100を維持
```

▼ 距離 dist / mag

```
let d = dist(x1, y1, x2, y2); // 2点間の距離

let m = mag(x1, y1); // 原点(0, 0)からの距離。ベクターで使用
```

▼ ArcTangent()

```
// (x0, y0)から(mouseX, mouseY)に向かう角度
let angle = atan2(mouseY - y0, mouseX - x0);
```

▼ Array関連

基本的にはJavaScriptのビルトインのアレイメソッドを使う
join(), indexOf(), concat(), shift(), unshift(), push(), pop(), reverse(), slice(), spliceなど

▼ シャッフル shuffle

```
let arr = [1, 2, 3];
let newArr = shuffle(arr); // シャッフルされ、arrは変更なし。
```

```
let newArr2 = shuffle(arr, true); // シャッフルされ、arr自身も変更される(newArr2 === arr)
```

▼ トランスフォーム

▼ 拡大・縮小 scale(w, h)

```
scale(2) // 2倍  
scale(2, 0.5) // x方向へ2倍、y方向へ0.5倍
```

▼ 座標原点の移動 translate(x, y)

```
translate(x, y)
```

▼ 座標原点を中心に回転 rotate(angle)

```
rotate(120); // デフォルトではdegree。120度  
  
angle = radians(120); // radianにできる  
rotate(angle);
```

▼ p5 Graphics(別のスクリーン)を作る

```
let pg;  
  
function setup() {  
  createCanvas(windowWidth, windowHeight);  
  
  pg = createGraphics(windowWidth/2, windowHeight/2); // pgという新しいスクリーン（キャンバス）を作成。w, hを設定。  
  pg.noFill(); // pgの初期設定  
  pg.stroke(0);  
  pg.imageMode(CENTER);  
  pg.background(255, 255, 255); // color  
}  
  
function draw() {  
  image(pg, width/2, height/2); // pgを描画。pgはひとつのイメージと考えればよい  
}
```

▼ イベント

```
// 全体に設定
function XXX(){ } // window.addEventListener('click', () => {})

// element (HTMLのエレメント) ごとに設定
Element.XXX(FUNC) // ELEMENT.addEventListener('click', FUNC);
```

▼ マウスイベント

▼ マウスダウン mouseIsPressed (boolean判定)

```
if(mouseIsPressed){};
```

▼ マウスダウン・アップ mousePressed() / mouseReleased()

```
function mousePressed(){};
function mouseReleased(){};

ELEMENT.mousePressed(FUNC);
ELEMENT.mouseReleased(FUNC);
```

▼ マウスムーブ mouseMoved()

```
function mouseMoved(){};

ELEMENT.mouseMoved(FUNC);
```

▼ マウスクリック mouseClicked()

```
function mouseClicked(){};

ELEMENT.mouseClicked(FUNC);
```

▼ ダブルクリック doubleClicked()

```
function doubleClicked(){};

ELEMENT.doubleClicked(FUNC);
```

▼ マウスオーバー、マウスリーブ mouseOver() / mouseOut()

```
function mouseOver(){};
function mouseOut(){};

ELEMENT.mouseOver(FUNC);
ELEMENT.mouseOut(FUNC);
```

▼ マウスドラッグ mouseDragged() // mousePressed & mouseMoved

```
function mouseDragged() {
  ellipse(mouseX, mouseY, 5, 5);
  // prevent default
  return false;
}
```

▼ キーイベント

▼ キー key

```
function keyCheck(){
  if(keyIsPressed){
    push();
    textSize(100);
    fill("#af0");
    text(key, width/2, 100);
    pop();
  }
}
```

▼ keyCode(数字) 、 <https://www.toptal.com/developers/keycode>

```
// The variable keyCode stores the
// BACKSPACE, DELETE, ENTER, RETURN, TAB, ESCAPE, SHIFT, CONTROL, OPTION,
// ALT, UP_ARROW, DOWN_ARROW, LEFT_ARROW, RIGHT_ARROW.
if (keyCode === UP_ARROW) {
  ///
}
```

```
window.addEventListener('keydown', e => {
  const keyCode = e.keyCode
})
```

▼ キープレス keyIsPressed (boolean判定)

```
//   if (keyIsPressed && (key == 'a')) {}など
```

▼ キーダウン KeyIsDown(KEY_CODE) (boolean判定)

```
// keyCodeの確認  
function keyPressed(){  
  console.log(keyCode);  
}
```

▼ キーダウン・アップ keyPressed() / keyReleased()

```
function keyPressed() {  
  if ((key == 'T') || (key == 't')) {  
    drawT = true;  
  }  
}  
  
function keyReleased() {  
  drawT = false;  
}
```

```
window.addEventListener('keydown', () => {})  
window.addEventListener('keyup', () => {})
```

▼ キータイプ関数 keyTyped(){}

```
function keyTyped() {  
  if (key === 'a') {  
    value = 255;  
  } else if (key === 'b') {  
    value = 0;  
  }  
}
```

▼ 複数キーを認識させるために

```
this.controlKeys = [83, 37, 38, 39, 40];  // 's', 'ArrowLeft', 'ArrowU  
p', 'ArrowRight', 'ArrowDown'  
this.keys = [];  
///  
keyControl(){  
  this.controlKeys.forEach(key => {
```

```

        if(keyIsDown(key) && !this.keys.includes(key)) this.keys.push
(key);
        if(!keyIsDown(key) && this.keys.includes(key)) this.keys.spli
ce(this.keys.indexOf(key), 1);
    })
}
///
if(this.keys.includes(this.controlKeys[0]) && frameCount % 10 === 0 && th
is.bullets.length < 5){
    this.bullets.push(new Bullet(bulletImage, this.x + this.w/2,
this.y));
    clickSound.play();
}

```

// 組み合わせで正しい挙動を示さない場合もある
スペース+アローアップ+アローレフトの組み合わせはなぜか動作しない => スペースではなく
's' を採用

▼ オーディオ

▼ 再生、停止などの操作

```

// p5.sound.jsをインポート

sound.play();
sound.pause();
sound.loop();    // ループ再生設定
sound.isPlaying(); // true or false
sound.stop();
sound.rate(rate); // 再生速度。0.5で半分のスピードで再生、2で2倍速で再生
sound.setVolume(v);
sound.currentTime(); // 現在までの経過時間
sound.duration(); // 全体の再生時間を取得
sound.jump(cueTime, duration); // cueTime: スタート位置 (秒)、duration: 再生時間

```

▼ p5.Amplitude() 音の大きさを数値で得る

```

let sound, amp;
// preload
sound = loadSound('sound.mp3');

// setup
amp = new p5.Amplitude();
amp.setInput(sound);

// draw

```

```
let level = amp.getLevel();  
// levelの値を使って画像を変形させるなどできる
```

▼ マイク

▼ 音量

```
let mic;  
  
// setup  
mic = new p5.AudioIn();  
mic.start();  
  
//draw  
level = mic.getLevel();  
// levelの値を使って画像を変形させるなどできる  
  
mic.stop();  
mic.enabled; // T/F オンになっているか
```

▼ 録音、保存

```
let mic, rec, sfile;  
  
// setup  
mic = new p5.AudioIn();  
rec = new p5.SoundRecorder();  
sfile = new p5.SoundFile();  
  
rec.setInput(mic);  
mic.start();  
  
////////////////////////////////////
```

▼ Local storage(ブラウザ上に保存されるデータ)

▼ データの取得

```
let data = getItem("MYDATA"); // "MYDATA"はローカルストレージ上のKey (この名前でデータが保存される)  
if (!data) { // "MYDATA"がnull (未だ存在しない) 場合  
  data = ''; // 文字列ではなく、array, objectでも可  
}
```

▼ データの保存


```
// let data = getItem("MYDATA");

// 文字列の場合
setItem('MYDATA', 'This is your text');

// arrayの場合
// data.push("something")やdata[0] = 99などの処理をした後に保存
setItem('MYDATA', data)

// objectの場合
// data["red"] = 100などの処理をした後に保存
setItem('MYDATA', data)
```

▼ データのクリア、削除

```
// 文字列の場合
setItem('MYDATA', 'This is your text');
// "MYDATA"は残したままクリア
clearStorage();

// "MYDATA"全体を削除
removeItem('MYDATA');
```

▼ HTMLエレメントの作成、操作など

▼ createElement()

```
// タグを指定で作成

let h5 = createElement('h5', 'hello'); // <h5>hello</h5>
h5.style("color", "#f0a"); // style
// h5.setAttribute(ATTR, VALUE)
h5.position(20, 20); // position: absolute;
```

▼ createDivなど

```
// Div, A, Spanなど様々

let div = createDiv("I am div element"); // <div>I am div element</div>
div.position(10, 10); // position: absolute;
div.style("font-size", "30px").("color", "red"); // styleの設定
// div.setAttribute(ATTR, VALUE)
```

```
let img = createImage(SRC, ALT);
img.position(0, 0);
```

▼ タグ内の文字列の変更、文字列の取得

```
let div = createDiv("How are you?"); // <div>How are you?</div>
// 文字列の変更
div.html('good');

// 文字列の取得
div.html();
```

▼ その他のDOM操作 hide(), show(), class(), addClass(), toggleClass(), id() など様々あり

```
let div = createDiv("How are you?"); // <div>How are you?</div>

div.hide();
div.show();
div.remove();
div.class('first'); // クラスを変更
div.addClass('second'); // クラスを追加。もしくはdiv.classList.add()
div.toggleClass('tog'); // mouseClickなどと組み合わせて使う
div.id('right');
// その他様々
// https://p5js.org/reference/#/p5.Element
```

▼ Elementの取得

```
let bodyElement = select("body");
// select("CSS SELECTOR") "#container"や".upper p"など
// getElementByIdやquerySelectorは機能しないことに注意

let divs = selectAll("div"); // すべてをアレイで取得
```

▼ ボタン createButton()

```
let button = createButton('click me');
button.html(LABEL); // 文字列を変更する場合に使う
button.size(w, h); // サイズを指定
button.position(0, 0); // 表示場所を指定
button.style(PROPERTY, VALUE); // "color", "red"
button.hide();
button.show();
```

```
button.mousePressed(FUNC); // 関数は引数無しで,()無しの名前にする

button.width;
button.height;
```

▼ セレクトプルダウンリスト createSelect()

```
select = createSelect(); // argumentにtrueとすることでマルチセレクトが可能になる
select.position(10, 10);
select.option('apple'); // argument二つ目でvalueを設定できる
select.option('grape');
select.selected('apple'); // 初期の選択、変更
select.disable('dog'); // 選択不可
select.changed(FUNC); // changed()はchangeイベントハンドラー
select.value(); // 選択されたオプションのValue。let item = select.value();
```

▼ ラジオボタン createRadio()

```
radio = createRadio();
radio.position(10, 10);
radio.option('100', 'apple'); // value, key
radio.option('200', 'grape');
radio.selected('100'); // 初期の選択
radio.disable('dog'); // 選択不可
radio.value(); // let val = radio.value();
```

▼ スライダー createSlider()

```
slider = createSlider(min, max, [currentValue], [step]);
slider.position(10, 10);
slider.style('width', '80px');

let val = slider.value();
```

▼ チェックボックス createCheckbox()

```
let checkbox1 = createCheckbox('label', false); // false => no check
checkbox1.changed(FUNC);
checkbox1.selected(value); // valueで指定された項目を返す???
checkbox1.checked() // true/false
checkbox1.value() // チェックされた値
checkbox1.disable();
```

▼ インプット createInput()

```
// input = createInput(value, [type]); // text, passwordなど
inputBox = createInput(DEFAULT_VALUE); // default値を設定できる
inputBox.position(20, 65);
inputBox.input(FUNC); // inputはinputイベントハンドラー

inputBox.changed(FUNC);

let val = inputBox.value();

// submitはcreateButtonをして処理を作成する
```

▼ ファイルインプット createFileInput()

```
// input = createInput(callback, [multiple]);
input = createFileInput(FUNC);
input.position(20, 65);
///
function FUNC(file){
  if(file.type === 'image'){ // "text", "video"など
    img = createImg(file.data, ''); // 文字列、base64エンコードデータ
    img.hide(); // hide(), display:none;
  } else {
    img = null;
  }
}
```

▼ イメージ createImg()

```
// <img>タグを生成
let img = createImg("pict.png", "picture"); // ふたつめのargumentはalt
img.position(100, 100);
```

▼ ノイズ noise(x, [y], [z]) / noiseSeed()

```
// Perlin noise
// 0 =< noise(x) =< 1の値
// xの変化量により変動幅を調整。小さい変化ほど変動は小さい（連続的）
let n = noise(x, [y], [z]);

noiseSeed(9); // いつも同じ動きをさせるときにはsetup()に設定しておく
```

▼ キャンバスの画像を保存

```
saveCanvas(canvas, 'my_picture', 'png') // 'jpg' or 'png'
```

▼ ベクトル p5.Vector

▼ p5.Vectorオブジェクトの作成

▼ createVector(x, y) / createVector(x, y, z)

```
let v = createVector(10, 20);  
let vx = v.x; // 10  
let vy = v.y; // 20
```

▼ p5.Vector.fromAngle(angle, length)

```
let v = p5.Vector.fromAngle(Math.PI / 6, 30);  
let vx = v.x;  
let vy = v.y;
```

▼ p5.Vector.random2D() / random3D

```
let v = p5.Vector.random2D(); // length == 1
```

▼ p5.Vector Methods

```
const v1 = createVector(100, 200);  
const v2 = createVector(80, 40);
```

▼ 合成・拡大 add() / sub() / mult() / setMag() / div()

```
v1.add(1, 2); // もしくはv.add([1, 2]); 101, 202に。v1自体が更新されることに注意  
v1.add(v2); // 180, 240に。v1自体が更新されることに注意  
  
let v3 = p5.Vector.add(v1, v2);
```

```
v1.sub(1, 2); // もしくはv.sub([1, 2]); 99, 198に。v1自体が更新されることに注意  
v1.sub(v2); // 20, 160に。v1自体が更新されることに注意
```

```
let v3 = p5.Vector.sub(v1, v2);
```

```
v1.mult(2);    // 200, 400に。v1自体が更新されることに注意  
v1.mult([1, 2]);    // 100, 400yに。それぞれ別の値で。v1自体が更新されることに注意  
v1.mult(v2);    // 8000, 8000に。v1自体が更新されることに注意  
  
let v3 = p5.Vector.mult(v1, 3);  
let v3 = p5.Vector.mult(v1, v2);
```

```
v1.setMag(10);    // lengthが10に。v1自体が更新されることに注意  
  
let v3 = p5.Vector.setMag(v1, 10);
```

```
v1.div(10);    // 10, 20。v1自体が更新されることに注意  
v1.div(v2);    // 1.25, 5に。v1自体が更新されることに注意  
  
let v3 = p5.Vector.div(v1, v2);
```

▼ アレイに v.array()

```
v1.array();    // [100, 200]
```

▼ 更新 v.set(x, y) / v.set(x, y, z)

```
v1.set(50, 100);  
console.log(v1);    // x, yが更新されたベクターに更新
```

▼ コピー v.copy()

```
const v3 = v1.copy();
```

▼ 大きさの取得 mag() / magSq()

```
v1.mag();  
p5.Vector.mag(v1)
```

```
// 三平方の定理の二乗のままのmagSq()は単純に大きさを比較する場合に用いる (早い)
```

▼ ノーマライズ (大きさを 1 に) `normalize()`

```
let v3 = v1.normalize(); // v1自体が更新されることに注意  
let v3 = p5.Vector.normalize(v1);
```

▼ 2ベクトルの距離 `dist()` // 正の値

```
let d = v2.dist(v1);  
let d = p5.Vector.dist(v1, v2)
```

▼ 大きさの制限 `limit()`

```
let v = v.limit(5) // v.limit(5)だけでも更新される
```

▼ 向きを取得・変更 `heading()` / `v.setHeading(angle)` or `v.rotate(angle)`

```
let angleNow = v1.heading(); // radian  
  
let v3 = v1.setHeading(PI); // PIラジアンに  
  
let v3 = v1.rotate(PI); // PIラジアン回転
```

▼ 2ベクトルの成す角 `angleBetween()`

```
let angle = v1.angleBetween(v2); // v1からv2への角
```

▼ 2ベクトルの内分点 `lerp()`

```
v1.lerp(v2, 0.5); // v1は内分するベクトルに置き換わる  
p5.Vector.lerp(v1, v2, 0.5);
```

▼ 判定 `equals()`

```
v1.equals(v2);
```